

Mapa del código de Lisart

En qué orden importan los archivos del sistema, qué se rompe si cada uno falla, y dónde termina la parte que decide *qué es verdad* y empieza la que decide *cómo se ve*. Es el documento de entrega para quien va a tomar el código.

Actualizada el 10 de septiembre de 2026

Este documento se genera desde `docs/mapa_del_codigo.md`. No se edita a mano: se regenera con `lisart guia codigo`.

Este documento contesta dos preguntas concretas: **¿por dónde empiezo?** y **¿qué puedo tocar sin romper nada?** El *qué hace* el sistema está en `documentacion_general.md`; el *cómo se opera*, en `guia_de_uso.md`; el detalle de cada tabla, en `modelo_de_datos.md`, que se genera solo.

La jerarquía de abajo **no es una opinión**: sale de medir qué módulos importan a cuáles. Ese número —de cuántos archivos depende cada archivo— está en la tabla de la sección 2, y es lo que decide cuánto daño hace un error en cada lugar.

Contenido

1. Cómo está ordenado esto, y por qué así

2. La jerarquía completa, en una tabla

3. Los siete niveles, uno por uno

4. Front-end y back-end: qué significan aquí

5. El reparto, archivo por archivo

6. Los cuatro archivos donde se mezclan, y por dónde pasa la costura

7. Las reglas que no se negocian

8. Por dónde empezar, si acabás de llegar

9. Qué se puede tocar con confianza y qué no

10. Las pruebas y el Makefile

11. Palabras que se usan aquí

12. Lo que hay que decirle a quien reciba esto

1. Cómo está ordenado esto, y por qué así

El criterio del orden es uno solo: **cuánto del sistema queda mal si este archivo está mal.** No es cuánto se usa, ni qué tan grande es, ni qué tan seguido se toca.

Esa distinción importa para quien recibe el código, porque los tres criterios apuntan a lugares distintos:

CRITERIO	ARCHIVO QUE GANA	QUÉ PASA SI FALLA
De cuántos depende todo	`tables.py`	Todo lo demás queda mal, en silencio
Cuánto se corre	`cli.py`	Truena a la vista, y no corrompe nada
Qué tan grande es	`ui/app.py`	Se ve feo o se cae la pantalla

El primero es el que manda aquí. **El daño silencioso es peor que el daño ruidoso**: un error en `cli.py` sale en la terminal y se corrige en diez minutos; un error en `ids.py` mete 72 personas duplicadas en la base y nadie se entera hasta que los recibos no cuadran.

Los archivos están agrupados en **siete niveles**. Dentro de cada nivel el orden también importa, pero lo que hay que memorizar son los niveles: un cambio en el nivel 0 se piensa dos veces y se prueba; un cambio en el nivel 6 se hace y ya.

2. La jerarquía completa, en una tabla

La columna **"lo importan"** es el dato duro: cuántos de los 24 módulos del paquete dependen de éste. Es la medida del radio de daño.

#	ARCHIVO	NIVEL	LO IMPORTAN	LÍNEAS	SI ESTO ESTÁ MAL…
1	`tables.py`	0 · Cimiento	**13**	695	Todo el sistema describe mal la realidad
2	`io.py`	0 · Cimiento	**17**	317	Los datos se leen o se escriben corruptos
3	`schema.py`	0 · Cimiento	8	174	No se puede describir una tabla bien
4	`config.py`	0 · Cimiento	11	105	Se escribe en la carpeta equivocada
5	`ids.py`	1 · Garantías	9	151	Se duplican personas **sin que nada avise**
6	`validate.py`	1 · Garantías	4	251	Entra basura y nadie la detiene
7	`backup.py`	1 · Garantías	6	131	Un error de captura ya no se puede deshacer
8	`fuentes/jotform.py`	2 · Entrada	2	654	Los datos entran en la columna equivocada
9	`procesos/alta_inscripciones.py`	2 · Entrada	1	698	La base de personas se construye mal
10	`fuentes/catalogos.py`	2 · Entrada	1	210	Los precios no llegan del Excel
11	`procesos/proyeccion.py`	3 · Valor	2	279	**La cifra de dinero sale mal**
12	`procesos/descuentos.py`	3 · Valor	2	463	Se cobra de más o de menos
13	`procesos/proyeccion_formato.py`	3 · Valor	1	487	El Excel de la escuela sale desacomodado
14	`procesos/horarios.py`	3 · Valor	3	324	Se ofrece una hora ocupada para reponer

#	ARCHIVO	NIVEL	LO IMPORTAN	LÍNEAS	SI ESTO ESTÁ MAL…
15	`procesos/roles.py`	3 · Valor	1	213	El directorio miente sobre quién es qué
16	`cli.py`	4 · Puerta	1	537	No se puede correr nada. **Truena a la vista**
17	`ui/app.py`	5 · Pantalla	1	1335	La pantalla no sirve; los datos siguen bien
18	`ui/grafo.py`	5 · Pantalla	1	624	El grafo se ve mal o no pinta
19	`ui/app_profesores.py`	5 · Pantalla	0	399	La agenda no sirve
20	`ui/semana.py`	5 · Pantalla	2	174	La semana se dibuja torcida
21	`ui/iconos.py`	5 · Pantalla	2	130	Faltan iconos
22	`assets/estilo.css`	5 · Pantalla	—	482	Se ve genérico
23	`vault.py`	6 · Documentos	1	768	El mapa de Obsidian sale mal
24	`guia.py`	6 · Documentos	1	424	Los PDF salen mal
25	`procesos/demo.py`	7 · Andamio	1	225	Los datos de prueba salen mal

Fuera de la tabla, y a propósito: `__init__.py` (24 líneas, sólo reexporta), `__main__.py` (8 líneas) y los `__init__.py` de `fuentes/`, `procesos/` y `ui/`. No deciden nada.

Dos lecturas que conviene hacer de esta tabla:

- **`io.py` tiene 17 dependientes y `cli.py` tiene 1.** No significa que `cli.py` importe poco: significa que nadie depende de él. `cli.py` está *arriba* de todo, `io.py` está *abajo* de todo. Lo de abajo se toca con guantes.
- **Las líneas no correlacionan con la importancia.** `ui/app.py` es el archivo más grande del proyecto (1335 líneas) y está en el nivel 5. `config.py` tiene 105 líneas y está en el nivel 0.

3. Los siete niveles, uno por uno

Nivel 0 — El cimiento

Cuatro archivos, y **son los que no se tocan sin pensarlo**. Todo lo demás se apoya aquí.

1. `tables.py` **— el modelo de datos.** Las 17 tablas se describen aquí, y de esa descripción salen solas la validación, la documentación técnica, el mapa de Obsidian y hasta los controles de la pantalla. Es el único archivo del que se puede decir: *si querés entender el sistema, empezá por aquí.*

> Tiene una dependencia rara que hay que conocer: **importa** `fuentes/jotform.py` para generar las 113 columnas de `respuestas_inscripcion` en vez de escribirlas a mano. Es la única vez que el cimiento depende de una fuente externa, y está hecho a propósito para que las columnas no se escriban dos veces.

2. `io.py` **— la única puerta a los datos.** Ningún otro archivo hace `pd.read_csv` ni `df.to_csv` sobre `data/`. Aquí viven los tipos, el orden de las filas y la escritura atómica. **Es la regla número uno del repositorio**, y la que más fácil se rompe por descuido al escribir un script nuevo.

3. `schema.py` **— el vocabulario.** Las piezas con las que `tables.py` describe una tabla: `Column`, `ForeignKey`, `Table`, `Registry`. Se toca sólo cuando hace falta un tipo de dato nuevo, que ha pasado una vez (el tipo `time`, para los horarios).

4. `config.py` **— dónde está cada cosa.** 105 líneas, ninguna dependencia, y lo importan 11 módulos. Rutas, formatos y la lectura de `.env`. Está en el nivel 0 porque un error aquí escribe los datos en el lugar equivocado, que es de los errores más caros de deshacer.

Nivel 1 — Las garantías

Lo que hace confiable al nivel 0. Ordenados por **qué tan silencioso es el daño** si fallan.

5. `ids.py` **— la identidad.** Genera las llaves y decide si dos menciones son la misma persona. Va primero de este nivel por una razón incómoda: **un error aquí pasa todas las validaciones**. Dos filas de una misma persona son dos filas perfectamente válidas. Es el error que no tiene quién lo atrape.

6. `validate.py` **— el guardia.** Seis revisiones: columnas, tipos, obligatorias, llave primaria, catálogo y referencias. Corre en cada escritura y con `lisart validate`.

7. `backup.py` **— la red.** Copia el estado anterior antes de cada escritura desde la pantalla, y sólo si el dato nuevo pasó la validación.

Nivel 2 — Por dónde entra la verdad

8. `fuentes/jotform.py` — 654 líneas, y el archivo con **el riesgo estructural más grande del sistema**: Jotform identifica los campos por su *posición*, así que reordenar una pregunta manda los datos a la

columna equivocada sin que nada truene. Por eso tiene `revisar_mapeo()`, que avisa en cada ingesta. Quien reciba el código tiene que saber esto antes que nada.

9. `procesos/alta_inscripciones.py` — 698 líneas: el proceso con más criterio del sistema. Convierte envíos en personas, alumnos y relaciones, y es donde vive la deduplicación y la regla de las fechas de nacimiento en conflicto.

10. `fuentes/catalogos.py` — lee el Excel de la escuela. El más sencillo de los tres.

Nivel 3 — Los procesos que producen valor

Aquí es donde conviene que trabaje alguien nuevo: **son los archivos que producen algo que a la escuela le importa, y ninguno puede corromper la base**, porque leen de `master/` y escriben en `exports/`.

11. `procesos/proyeccion.py` es el más visible: es la cifra mensual. **12.** `procesos/descuentos.py` mantiene el archivo que se edita en Excel. **13.** `procesos/proyeccion_formato.py` arma el Excel con el formato de la escuela. **14.** `procesos/horarios.py` contesta cuándo puede alguien y qué huecos comparten un profesor y un alumno. **15.** `procesos/roles.py` es el más chico y el mejor ejemplo del patrón: lógica de dominio en un módulo propio, y la pantalla sólo lo llama.

Nivel 4 — La puerta

16. `cli.py` — los 13 comandos. Es lo que más se corre de todo el sistema y aun así está en el nivel 4: no guarda ninguna verdad, sólo orquesta. Un error aquí es ruidoso y barato. **Es el mejor lugar para el primer cambio de alguien que llega**: se ve el efecto de inmediato y no puede romper datos.

Nivel 5 — La pantalla

17. `ui/app.py` (1335 líneas, la más grande), **18.** `ui/grafo.py`, **19.** `ui/app_profesores.py`, **20.** `ui/semana.py`, **21.** `ui/iconos.py` y **22.** `assets/estilo.css`.

Si toda esta capa desapareciera, el sistema seguiría funcionando por línea de comandos y **ni un dato se perdería**. Eso es exactamente lo que la pone en el nivel 5. Lo que sí se perdería es que alguien no técnico pueda usarlo, que no es poco.

Nivel 6 — Los documentos que se generan

23. `vault.py` genera el mapa de Obsidian; **24.** `guia.py` genera los tres PDF de `docs/`. Producen documentación, no datos. Un error aquí se ve al abrir el archivo.

Nivel 7 — El andamio

25. `procesos/demo.py` siembra datos de prueba marcados con `fuente="demo"`. Se borran con una orden sin tocar un dato real. Es lo último de la lista y está bien que lo sea: **es el único archivo que se podría borrar sin consecuencias** —al costo de no poder volver a demostrar la agenda hasta que haya profesores reales.

4. Front-end y back-end: qué significan aquí

Antes del reparto hay que decir algo, porque la respuesta ingenua — *`ui/` es el front-end y lo demás el back-end* — es corta y confunde a quien llega.

En este proyecto no hay un front-end clásico. No hay JavaScript, no hay una aplicación de navegador, no hay una API entre las dos mitades. Streamlit corre **Python en la computadora** y manda al navegador el resultado ya pintado. Así que las dos palabras hay que redefinirlas:

	QUÉ ES AQUÍ
Back-end	El código que decide **qué es verdad**: qué dato entra, cómo se guarda, cuánto se cobra
Front-end	El código que decide **cómo se ve**: SVG, HTML, CSS, controles, y el texto que se imprime

La línea real no es la carpeta `ui/`: es `io.py`. Lo que toca `data/` es back-end, sin importar dónde viva. Y ahí está el detalle que hay que señalarle a quien reciba esto: hay archivos en `ui/` que **sí** tocan datos (sección 6).

Tres cosas más, propias de este proyecto:

1. **El front-end está escrito en Python y se manda como texto.** El grafo, el pentagrama y la semana son SVG que se arma con `f-strings`. No es capricho: **ninguna librería de gráficas dibuja nada en este equipo** —plotly, altair y los gráficos nativos salen vacíos, sin error—. Está comprobado y documentado. Quien llegue va a querer "arreglarlo" instalando una librería: no funciona.
2. **La terminal también es un front-end.** Todo lo que `cli.py` imprime lo lee una persona cada semana. Es la interfaz que más se usa del sistema.
3. **Hay front-end que no corre frente a nadie.** `guia.py` escribe HTML y CSS para que Chromium lo imprima en PDF, y `vault.py` escribe markdown para Obsidian. Es código de presentación, pero se ejecuta una vez y deja un archivo.

De las ~9,800 líneas de Python del paquete, **algo así como una cuarta parte decide cómo se ve algo** (más las 482 líneas de CSS, que son puro aspecto). Es una estimación, no una cuenta exacta: en tres archivos las dos cosas conviven en la misma función.

5. El reparto, archivo por archivo

ARCHIVO	BACK-END	FRONT-END	QUÉ HACE DE CADA LADO
`tables.py`	**Todo**	—	Describe la realidad. De aquí salen los controles, pero no los dibuja
`io.py`	**Todo**	—	Lee y escribe. Es la línea divisoria en persona
`schema.py`	**Todo**	—	El vocabulario del modelo
`config.py`	**Todo**	—	Rutas y formatos
`ids.py`	**Todo**	—	Llaves e identidad
`validate.py`	Casi todo	Un poco	Revisa (back); redacta el mensaje de error que lee una persona (front)
`backup.py`	**Todo**	—	Copias
`fuentes/jotform.py`	**Todo**	—	Baja, aplana, avisa
`fuentes/catalogos.py`	**Todo**	—	Lee el Excel
`procesos/alta_inscripciones.py`	**Todo**	—	Deduplica y construye
`procesos/proyeccion.py`	**Todo**	—	Calcula. Escribe CSV, que es dato, no diseño
`procesos/descuentos.py`	**Todo**	—	Resuelve y reporta
`procesos/horarios.py`	**Todo**	—	Estados, rangos y reposición
`procesos/roles.py`	**Todo**	—	Directorio y vigencias
`procesos/demo.py`	**Todo**	—	Siembra
`procesos/proyeccion_formato.py`	Mitad	Mitad	Calcula las columnas (back); acomoda el Excel de la escuela (front, en otro medio)
`cli.py`	Mitad	Mitad	Orquesta los procesos (back); **todo lo que imprime es interfaz** (front)
`ui/grafo.py`	Mitad	Mitad	Lee datos y calcula el acomodo (back); dibuja el SVG (front)
`ui/app.py`	Un tercio	Dos tercios	Pinta la pantalla (front); **guarda, filtra y genera IDs** (back)
`ui/app_profesores.py`	Un tercio	Dos tercios	Igual, más chico: también **crea clases y asigna profesores**

ARCHIVO	BACK-END	FRONT-END	QUÉ HACE DE CADA LADO
`ui/semana.py`	Un poco	Casi todo	Calcula la ventana horaria (back); dibuja las barras (front)
`ui/iconos.py`	—	**Todo**	SVG y nada más
`assets/estilo.css`	—	**Todo**	El sistema de diseño
`vault.py`	Un poco	Casi todo	Lee el modelo (back); escribe markdown para Obsidian (front)
`guia.py`	Un poco	Casi todo	Huellas y registro de documentos (back); HTML y CSS de imprenta (front)

6. Los cuatro archivos donde se mezclan, y por dónde pasa la costura

Estos son los que hay que explicar en persona a quien reciba el código. En los demás la respuesta es obvia; en éstos, no.

`ui/app.py` — front-end con back-end adentro

La mayor parte pinta. Pero hay funciones que **escriben en `master/`** y que, si se rompen, corrompen datos de verdad:

FUNCIÓN	QUÉ HACE, Y POR QUÉ ES BACK-END
`_guardar`	Valida, respalda y escribe la fila. Es el camino de escritura de la pantalla
`_agregar_bloque`	Genera un ID nuevo y escribe en `disponibilidad`
`_filtrar`	Decide qué registros se ven al buscar
`_titulo_registro`	Arma el nombre legible cruzando tablas
`tiene_valor`	La función que evita el error de pandas con celdas vacías

> **La regla al tocar este archivo:** si lo que vas a cambiar decide *qué se guarda*, no va aquí — va a un módulo de `procesos/` y la pantalla lo llama. Eso ya se hizo una vez, con los roles (`procesos/roles.py`), y salió bien: la lógica quedó probada sin levantar Streamlit, con 11 pruebas rápidas.

`ui/grafo.py` — la mitad exacta

Es el ejemplo más limpio de las dos naturalezas en un archivo, y la costura está marcada en el propio código:

BACK-END	FRONT-END
`construir()` — arma el grafo desde las tablas	`_figura()` — dibuja una nota musical
`_posiciones()` , `_escalar()` , `_separar()` — el acomodo	`_arista()` — dibuja un arco
`detalle()` , `parecidos()` — qué se sabe de un nodo	`svg()` — pega todo
	`leyenda()`

Ojo con un detalle: **`ui/grafo.py` importa `io` directamente**, o sea que un archivo de pantalla lee datos. Funciona y está acotado, pero es la única vez que pasa en la carpeta `ui/` fuera de `app.py` .

`cli.py` — la interfaz que no parece interfaz

Cada `cmd_*` hace dos cosas: llama al proceso (back) y **redacta lo que ve la persona** (front). Ese texto no es accesorio: es la interfaz que la escuela usa cada semana, y ahí es donde el sistema explica lo que no pudo hacer. La función `ruta_corta()` existe sólo para que las rutas se impriman legibles.

`procesos/proyeccion_formato.py` — presentación en Excel

Calcula columnas (back) y las acomoda en el formato de la escuela con sus anchos y sus totales (front, pero en un archivo, no en pantalla). Diseño es diseño aunque el medio sea una hoja de cálculo.

7. Las reglas que no se negocian

Cinco. Están en `CLAUDE.md` y en `docs/convenciones_datos.md`, y son las que sostienen todo lo anterior.

1. **Nunca `pd.read_csv` ni `df.to_csv` sobre `data/`.** Todo pasa por `read_table` / `write_table` / `upsert` de `lisart.io`.
2. **Las tablas se definen sólo en `tables.py`.** Después de tocarlo: `make docs` y `lisart validate`.
3. **Los catálogos no se llenan desde el código.** Si llega un valor que no está en el catálogo, se **avisa** y no se escribe nada.
4. **Nada se sobrescribe sin respaldo.**
5. **Nunca evaluar una celda de pandas como booleano directo.** `if fila["x"]` con una celda vacía revienta. Usar `ui.app.tiene_valor()` o `pd.isna()`.

Y antes de dar por bueno cualquier cambio: `make check` (ruff + pruebas + `lisart docs --check` + `lisart validate` + `lisart descuentos --revisar` + `lisart guia --check`).

8. Por dónde empezar, si acabás de llegar

Un orden de lectura probado, de menos a más riesgo. No intentes leer los 9,800 renglones: no hace falta.

Primero (una hora). `tables.py`, y nada más. Con las 17 tablas en la cabeza, el resto del código se explica solo. Ayuda tener abierto `docs/modelo_de_datos.md`, que sale de ahí.

Segundo (una hora). `io.py` y `schema.py`, en ese orden. Ahí se entiende por qué el proyecto insiste tanto en que nadie más toque `data/`.

Tercero (media hora). Correr las cosas: `lisart validate`, `lisart describe`, `lisart proyeccion`, y abrir `lisart ui`. Ver el sistema funcionando antes de leer un proceso.

Cuarto. Un proceso completo de punta a punta, y que sea `procesos/roles.py` (213 líneas): es el más chico y el que mejor muestra el patrón —dominio en `procesos/`, pantalla que sólo llama—.

Quinto. Ya con eso, `procesos/proyeccion.py` y `procesos/descuentos.py`, que son los que producen lo que a la escuela le importa.

Al final, y sólo si hace falta: `fuentes/jotform.py` y `procesos/alta_inscripciones.py`. Son los dos más largos y los más delicados, y casi nunca hay que tocarlos: se leen cuando algo del formulario cambió.

Para el primer cambio propio: algo en `cli.py` o en un proceso del nivel 3. Se ve el efecto de inmediato y no puede corromper la base.

9. Qué se puede tocar con confianza y qué no

ZONA	RIESGO	ANTES DE TOCAR
`procesos/`, `cli.py`	**Bajo.** Leen de `master/`, escriben en `exports/`	Correr `make check`
`ui/`, `assets/estilo.css`	**Bajo**, salvo las funciones de la sección 6	Mirarlo: capturas, no sólo pruebas
`vault.py`, `guia.py`	**Bajo.** Sólo generan documentos	Abrir el archivo generado
`validate.py`, `backup.py`	**Medio.** Aflojar una revisión deja pasar basura	Agregar una prueba que falle antes del cambio
`fuentes/`	**Medio.** Depende de sistemas de fuera	Probar con `--desde-raw`, sin llamar a la API
`ids.py`	**Alto.** El error no lo atrapa nadie	Pensar qué pasa con los datos que ya existen
`io.py`, `schema.py`	**Alto.** Lo importan casi todos	`make test-todo`, no sólo las rápidas
`tables.py`	**Alto.** Cambiar una llave con datos cargados es una migración	`make docs` + `lisart validate`, y leer el nivel 0

Una advertencia concreta: cambiar el nombre o el tipo de una columna que ya tiene datos **no es un cambio de código, es una migración**. Hoy hay 194 filas apuntando a llaves de `personas` y 129 a `instrumentos`. Renombrar una tabla salió gratis una vez (`salones` → `espacios`) sólo porque estaba vacía.

10. Las pruebas y el `Makefile`

No están en la jerarquía a propósito, y no porque importen poco: **no son el sistema, son lo que impide que el sistema se rompa sin avisar.** Son 208 pruebas en 13 archivos, y su valor está en que cubren justo lo que a alguien nuevo se le puede ocurrir cambiar.

ARCHIVO	QUÉ PROTEGE
`test_io.py`, `test_validate.py`	El cimiento: tipos, llaves, referencias
`test_ids.py`	La identidad, y el error de pandas con celdas vacías
`test_alta_inscripciones.py`	La deduplicación y las fechas en conflicto
`test_jotform.py`	El aplanado y el aviso de mapeo
`test_proyeccion.py`, `test_descuentos.py`	Que el dinero salga bien
`test_horarios.py`, `test_roles.py`	Los procesos del nivel 3
`test_backup.py`, `test_vault.py`, `test_guia.py`	Respaldos y documentos
`test_ui.py`	La pantalla. **Tardan ~5 min**, por eso llevan marcador `ui`

El ciclo rápido es `make test` (sin las de pantalla, ~2 min); `make test-todo` las incluye. Y `make check` es la revisión completa, la que tiene que pasar antes de dar cualquier cosa por buena.

11. Palabras que se usan aquí

PALABRA	QUÉ QUIERE DECIR
Módulo	Un archivo `.py`. Aquí también se les dice *scripts*
Paquete	La carpeta `src/lisart/` completa
Importar	Que un archivo use el código de otro. Es lo que se midió para la jerarquía
Dependencia	Lo que un archivo necesita para funcionar
Radio de daño	Cuánto del sistema queda mal si un archivo falla
Capa	Una etapa del camino del dato: `raw`, `interim`, `master`, `exports`
Back-end	El código que decide qué es verdad
Front-end	El código que decide cómo se ve
SVG	Un dibujo escrito como texto. Así están hechos el grafo y la semana
Migración	Cambiar la forma de datos que ya existen. Nunca es sólo código
`make check`	La revisión completa que tiene que pasar antes de entregar

12. Lo que hay que decirle a quien reciba esto

Cinco frases, si sólo hay tiempo para cinco:

1. **Empezá por `tables.py`.** Todo lo demás sale de ahí.
2. **Nadie toca `data/` más que `io.py`.** Esa es la regla que sostiene el resto.
3. **Un error en `ids.py` no lo atrapa nadie.** Es el único lugar donde el daño es completamente silencioso.
4. **Lo que decide qué se guarda no vive en `ui/`.** Si estás escribiendo lógica de datos dentro de la pantalla, va en `procesos/`.
5. **`make check` antes de entregar**, siempre. Y si el cambio se ve, además mirarlo: hay defectos que ninguna prueba detecta.